Self-Paced Module: Swing Basics

Java Programming · Grade 12 TVL · Cielito Zamora Senior High School
Week 17 · November 9–13, 2026 · Continues Week 16 Threads

Name: _____ Section: _____ Date: _____

Day / time	Topic	Work and strip
Mon Nov 9 / 2 h	Swing; window, label, EDT	Page 1 Step 1; Monday strip, page 7 bottom.
Tue Nov 10 / 2 h	Fields, buttons, listeners	Page 2 Step 2; Tuesday strip, page 7 middle.
Wed Nov 11 / 2 h	Vehicle rules; validation	Page 3 Steps 3–4; Wednesday strip, page 7 top.
Thu Nov 12 / 3 h	Responsive interface; own-theme plan	Page 4 Step 5, then page 6 task; Thursday strip, page 8 bottom.
Fri Nov 13 / 3 h	Own-theme task and presentations	Page 6 Step 6/task; Friday strip, page 8 top.

How to use. Type and run in order. Predict, compare actual results, and correct mistakes. LIPLET means quiz strip; detach only today's strip and keep the rest until Friday. No internet needed.

JDK only. Put each class in its own named file, all in one folder. Compile `javac *.java`; run `java Main` on a computer with a display. Step 4 uses `java FareCheck`. Error Guide: page 5. Close the old window before rebuilding.

Where we left off

Files	What to do
Vehicle, Jeepney, Tricycle, Bus; OverloadException, InvalidDistanceException, VehicleNotFoundException (all .java)	Retain unchanged; 7 model/exception files from Appendix A.
DispatchThread, BoardingTask, ReadyReport, ConvoyLeg, LoadingBay (all .java)	Retain unchanged; 5 Week 16 helper files. They are not called by this week's GUI.
Main.java	Copy to Main_week16.txt first. Replace at Steps 1 and 5. Never keep two public Main classes as .java files.

MONDAY | From Console to Window: Swing and Safe Startup

Definition. A graphical user interface (GUI) lets a person use windows and controls with a keyboard or mouse. Swing is Java's built-in toolkit for these controls. AWT (Abstract Window Toolkit) supplies older controls, window support, events, and layout tools; Swing builds on AWT [NTU].

> **ANALOGY** The terminal window is the service counter. A label is the sign above it. The event dispatch thread is the clerk who changes signs and handles requests, one short job at a time.

The **event dispatch thread (EDT)** creates and updates the interface and handles its actions. invokeLater queues a job there [NTU] [NUS].

STEP 1 Show the terminal

Back up Main.java as Main_week16.txt. Replace Main.java and add TerminalPanel.java. Compile with javac *.java; run java Main.

Main.java

```
import java.awt.GraphicsEnvironment;
import javax.swing.*;
public class Main {
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println("No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        JFrame frame = new JFrame("Vehicle Terminal");
        frame.setDefaultCloseOperation(
          JFrame.EXIT_ON_CLOSE);
        frame.setContentPane(new TerminalPanel());
        frame.setSize(720, 180);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
      }
    });
  }
}
```

TerminalPanel.java

```
import javax.swing.*;
public class TerminalPanel extends JPanel {
  public TerminalPanel() {
    add(new JLabel("NXA 123 | Mang Ramon"));
    add(new JLabel("Monumento-Divisoria"));
  }
}
```

JFrame is the window; JLabel is its sign; JPanel groups the signs. Main shows the panel inside the frame.

School-display check: verify the Vehicle Terminal title, both labels, and closing the window. These are expected checks; appearance was not observed in this workspace.

Actual run without a display: No display: GUI not opened. This headless run has no window display.

> **Check yourself.** Change the first label to NYB 321 | Mang Tonyo. Predict and verify the changed text; the title is unchanged. Restore it.

Sources [1 NTU] [2 HWS] [3 MIT] [4 CMU] [5 NUS] (p5). Key validated facts: A JFrame contains the interface [HWS] [CMU]; create and update it on the event dispatch thread [NTU] [NUS].

> **LIPLET · MONDAY** After Step 1 on page 1, answer the MONDAY strip (bottom of page 7) in pen. Tear off only today's strip and submit it.

TUESDAY | A Click Has a Job: Buttons, Text Fields, and Listeners

Definition. An event is a report that something happened, such as a click. An ActionListener is an object whose actionPerformed method responds to that report. Register it with addActionListener; otherwise the button has no action [HWS] [MIT].

> **ANALOGY** A button is the bell at a sari-sari store. Ringing it is the event; the listener is the shopkeeper's agreed response. A bell with no response plan makes noise but serves nobody.

Control / method	Picture it as	Use it for
JTextField / getText	The clerk's input pad	Read a String, even when it contains digits.
JButton / addActionListener	Bell plus response plan	Register what happens after a click.
JLabel / setText	A sign that can change	Show a result or helpful feedback.

STEP 2 Respond to a click

Replace TerminalPanel.java only. Retain Monday Main.java. Compile with javac *.java; run java Main.

TerminalPanel.java

```
import java.awt.event.*;
import javax.swing.*;
public class TerminalPanel extends JPanel
    implements ActionListener {
  private JTextField plate =
    new JTextField("NXA 123", 8);
  private JLabel result =
    new JLabel("Waiting for a click");
  public TerminalPanel() {
    JLabel label = new JLabel("Plate:");
    label.setLabelFor(plate);
    JButton show = new JButton("Show");
    show.addActionListener(this);
    add(label);
    add(plate);
    add(show);
    add(result);
  }
  @Override public void actionPerformed(
      ActionEvent e) {
    result.setText("Plate: " + plate.getText().trim());
  }
}
```

Read the new syntax. `implements ActionListener` promises an actionPerformed method. `this` means this panel object; it is the registered listener. `@Override` asks the compiler to check that the promised method matches. `ActionEvent e` carries the event; this first example does not need its details.

A click in order	What happens
1. Type	The field holds TRC 456 as text.
2. Click Show	Swing reports an action event on the EDT.
3. Registration connects it	Swing calls this panel's actionPerformed.
4. Read and display	getText reads it; trim removes outside spaces; setText changes the label.

Observed console result

```
No display: GUI not opened.
```

School-computer action check: enter TRC 456 and click Show; the label should read Plate: TRC 456. Change it to BUS 789 and click again. These are expected action results for you to verify, not observed window results.

> **Check yourself.** enter TRC 456, then click Show; the result should read Plate: TRC 456. Try blank text: this early example accepts it. Explain why Wednesday needs validation. Use Tab to reach Show, then Space to activate it; record what you actually observe.

Pair practice and access check

Partner A traces the four steps; Partner B points to the code, then swap. setLabelFor links a prompt to its input. Test Tab and Space on the school display.

Spot the limit. This version can display any text, including an empty plate. It does not know whether a vehicle exists. Tomorrow we will call the existing Vehicle program and refuse bad input. Do not "fix" this by putting fare formulas into the listener.

Save locally: only TerminalPanel.java changes today. If Git is installed, commit `week17-day2-action-listener` and upload later.

Sources [1 NTU] [2 HWS] [3 MIT] [4 CMU] [5 NUS] (p5). Key validated facts: A click creates an event, registration connects the listener, and the listener changes a component [MIT] [HWS] [NTU].

> **LIPLET · TUESDAY** After Step 2 on page 2, answer the TUESDAY strip (middle of page 7) in pen. Tear off only today's strip and submit it.

WEDNESDAY | Reuse the Vehicle Program: Valid Input and Fare Results

Definition. The model holds data and rules: here, the existing Vehicle objects. The interface reads input, calls the model, and displays a result or helpful error [HWS] [NTU].

ANALOGY The service counter does not invent the fare. It asks the existing fare rules, just as our Swing panel asks Vehicle.quoteFare. A misspelled plate should receive a clear reply.

Validation checks input before accepting it. Refuse unknown plates and non-numbers. Distance must be finite (neither NaN, “not a number,” nor infinity) and above zero. Fares are classroom practice values, not LTFRB rates.

STEP 3 Connect the existing Vehicle objects

Add FareService.java; replace TerminalPanel.java. Retain Main.java; compile and run as before.

FareService.java

```
import java.util.Locale;
public class FareService {
  public static String quote(
      String plate, String distance)
      throws VehicleNotFoundException {
    double km =
      Double.parseDouble(distance.trim());
    if (!Double.isFinite(km) || km <= 0) {
      throw new InvalidDistanceException(
        "Use a finite km above 0.");
    }
    Vehicle[] fleet = {
      new Jeepney("NXA 123", "Mang Ramon",
        "Monumento-Divisoria"),
      new Tricycle("TRC 456", "Kuya Ben",
        "Bagong Silang TODA"),
      new Bus("BUS 789", "Aling Rosa", 50),
      new Jeepney("NYB 321", "Mang Tonyo",
        "Sangandaan-Recto")
    };
    String key = plate.trim()
      .toUpperCase(Locale.ROOT);
    for (Vehicle v : fleet) {
      if (v.getPlateNumber().equals(key)) {
        double fare = v.quoteFare(km);
        if (!Double.isFinite(fare)) {
          throw new InvalidDistanceException(
            "Distance is too large.");
        }
        return String.format(Locale.ROOT,
          "%s: P%.2f", key, fare);
      }
    }
    throw new VehicleNotFoundException(
      "Unknown plate: " + key);
  }
}
```

TerminalPanel.java

```
import java.awt.event.*;
import javax.swing.*;
public class TerminalPanel extends JPanel
    implements ActionListener {
  private JTextField plate =
    new JTextField("NXA 123", 8);
  private JTextField km = new JTextField("5", 5);
  private JLabel result =
    new JLabel("Enter plate and km");
  public TerminalPanel() {
    JLabel p = new JLabel("Plate:");
    JLabel d = new JLabel("Distance (km):");
    p.setLabelFor(plate);
    d.setLabelFor(km);
    JButton quote = new JButton("Quote fare");
    quote.addActionListener(this);
    add(p); add(plate); add(d); add(km);
    add(quote); add(result);
  }
  @Override public void actionPerformed(
      ActionEvent e) {
    try {
      result.setText(FareService.quote(
        plate.getText(), km.getText()));
    } catch (NumberFormatException ex) {
      result.setText("Enter a number for km.");
    } catch (VehicleNotFoundException ex) {
      result.setText(ex.getMessage());
    } catch (InvalidDistanceException ex) {
      result.setText(ex.getMessage());
    }
  }
}
```

STEP 4 Check real fare results

Add FareCheck.java. Compile as before; run java FareCheck. No display is needed.

FareCheck.java

```
public class FareCheck {
  public static void main(String[] args) {
    String[][] cases = {{"NXA 123", "5"}, {"TRC 456", "2"},
      {"BUS 789", "4"}, {"NYB 321", "6"}, {"nxA 123", " 5 "},
      {"ZZZ", "5"}, {"NXA 123", "abc"}, {"NXA 123", "0"},
      {"NXA 123", "NaN"}, {"NXA 123", "1e309"}};
    for (String[] c : cases) {
      try {
        System.out.println(
          FareService.quote(c[0], c[1]));
      } catch (VehicleNotFoundException |
        NumberFormatException | InvalidDistanceException e) {
        System.out.println(
          e.getClass().getSimpleName() + ": "
          + e.getMessage());
      }
    }
  }
}
```

Observed console result

```
NXA 123: P15.00
TRC 456: P40.00
BUS 789: P25.00
NYB 321: P17.00
NXA 123: P15.00
VehicleNotFoundException: Unknown
plate: ZZZ
NumberFormatException: For input
string: "abc"
InvalidDistanceException: Use a finite
km above 0.
InvalidDistanceException: Use a finite
km above 0.
InvalidDistanceException: Use a finite
km above 0.
```

Check yourself. Try BUS 789 / 4: expect P25.00. Correct abc to 4. InvalidDistanceException extends RuntimeException directly; catch it separately.

Sources [1 NTU] [2 HWS] [3 MIT] [4 CMU] [5 NUS] (p5). Key validated facts: Text input needs conversion [NTU]; keep event handling connected to the existing model and perform interface access on the event thread [HWS] [NUS].

LIPLET · WEDNESDAY After Step 4 on page 3, answer the WEDNESDAY strip (top of page 7) in pen. Tear off only today’s strip and submit it.

THURSDAY | Keep the Window Responsive: Background Work and Safe Updates

Definition. A responsive interface continues to handle actions while work is in progress. A background thread does the slow job. The event dispatch thread (EDT), introduced Monday, handles interface actions and updates. SwingUtilities.invokeLater queues an update on the EDT [NTU] [NUS].

ANALOGY The clerk asks a helper to check a loading bay. The clerk can still answer other customers. The helper returns a report; only the clerk changes the counter sign.

On the EDT	On the worker
Read fields; disable Check ready; show Checking...; return quickly.	Wait 1.5 seconds (simulated slow work), then queue a report.
Later: set readiness text and restore the button.	Never read fields or directly call setText on a visible control.

STEP 5 Wait without freezing the interface

Add ReadyTask.java and replace Main.java. Retain Wednesday TerminalPanel.java and FareService.java. Compile with javac *.java; run java Main.

Main.java

```
import java.awt.GraphicsEnvironment;
import java.awt.event.*;
import javax.swing.*;
public class Main {
  public static void main(String[] args) {
    if (GraphicsEnvironment.isHeadless()) {
      System.out.println(
        "No display: GUI not opened.");
      return;
    }
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        JFrame frame =
          new JFrame("Vehicle Terminal");
        frame.setDefaultCloseOperation(
          JFrame.EXIT_ON_CLOSE);
        TerminalPanel panel = new TerminalPanel();
        JLabel status = new JLabel("Not checked");
        JButton check =
          new JButton("Check ready");
        check.addActionListener(
          new ActionListener() {
          public void actionPerformed(
            ActionEvent e) {
            check.setEnabled(false);
            status.setText("Checking...");
            new Thread(
              new ReadyTask(status, check),
              "Readiness").start();
          }
        });
        panel.add(check); panel.add(status);
        frame.setContentPane(panel);
        frame.setSize(720, 220);
        frame.setLocationRelativeTo(null);
        frame.setVisible(true);
      }
    });
  }
}
```

ReadyTask.java

```
import javax.swing.*;
public class ReadyTask implements Runnable {
  private JLabel status;
  private JButton button;
  public ReadyTask(
      JLabel status, JButton button) {
    this.status = status;
    this.button = button;
  }
  @Override public void run() {
    String message;
    try {
      Thread.sleep(1500);
      message = "Ready: NXA 123";
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      message = "Check interrupted; try again.";
    }
    final String outcome = message;
    SwingUtilities.invokeLater(new Runnable() {
      public void run() {
        status.setText(outcome);
        button.setEnabled(true);
      }
    });
  }
}
```

What happens and why

The inner listener captures status and check: it keeps access to those local variables. Java 8 allows this because they are not reassigned. The final outcome value also cannot be reassigned.

The worker uses Week 16 Runnable and start(). It sleeps to simulate a slow check, then queues a Runnable on the EDT to update the sign and restore the button.

Interruption is a stop request. Restore its flag and report the interruption. Never make the EDT wait by calling join, await, or sleep in a listener.

Observed console result

```
No display: GUI not opened.
```

School-computer test: click Check ready. Expect Checking... and a disabled button, followed by Ready: NXA 123 and an enabled button. While waiting, quote BUS 789 at 4 km; expect P25.00 without waiting for readiness. Record the actual behavior.

Check yourself. while Check ready is running, try Quote fare. The fare action should still work; the ready button should be disabled until the update. Do not call join, await, or sleep in the listener. Record whether your school computer passes this test.

Sources [1 NTU] [2 HWS] [3 MIT] [4 CMU] [5 NUS] (p5). Key validated facts: Workers can queue interface updates using invokeLater [NTU] [NUS]; the window remains active after main ends [HWS].

LIPLET · THURSDAY After Step 5 on page 4, answer the THURSDAY strip (bottom of page 8) in pen. Tear off only today's strip and submit it.

Thursday next: after the strip, use the final hour for the plan and first working own-theme action on page 6. Keep all old files; only Main.java changes again today.

ERROR GUIDE | Offline help and honest checks

Actual message (excerpt)	What it means	Fix
error: cannot find symbol; symbol: class JFrame	The Swing class has not been imported.	Add import javax.swing.*; above the class. Check spelling and capitalization.
error: unreported exception VehicleNotFoundException; must be caught or declared to be thrown	A checked exception from FareService.quote is not caught.	Catch VehicleNotFoundException in the listener, as Step 3 shows. Do not add throws to main to hide the problem.
NumberFormatException: For input string: "abc"	Text cannot be parsed as a number.	Catch the input error; tell the user to enter a number and try again.
InvalidDistanceException: Use a finite km above 0.	0, a negative number, NaN, or infinity was given.	Enter a finite positive distance. Keep the model and interface checks.
VehicleNotFoundException: Unknown plate: ZZZ	The plate is not one of the four demonstration vehicles.	Use a known plate, or add a real vehicle entry deliberately.
Exception in thread "main" java.awt.HeadlessException	An unguarded JFrame was created without a display.	Run on a computer with a display. Main reports No display instead in headless mode.

Compiler wording above was produced with JDK 21 using Java 8 compatibility checks; the API-compatible programs use only Java 8 features. Compiler line numbers and newer-JDK warnings may differ. Each runtime message was produced by an actual run. The HeadlessException came from a separate unguarded display probe.

Keep the model: Vehicle is abstract; Jeepney, Tricycle, and Bus supply their own fare and capacity. FareService calls quoteFare rather than copying formulas. VehicleNotFoundException and OverloadException are checked; InvalidDistanceException is unchecked. Boarding is not used in the fare action. Locale.ROOT fixes decimal formatting independently of computer language.

When no error message appears

Symptom	First check
Window does not appear	Look behind other windows; check setVisible(true). A display-server connection error is an environment problem, not a fare rule.
Click does nothing	Check addActionListener and method spelling. Recompile after saving.
Window freezes during slow work	Find sleep, join, await, or slow file work inside a listener; move the job to a background thread.
Text or controls are hard to read	Resize on the school display. Keep clear labels. Report clipping instead of claiming an unobserved appearance is correct.

What was actually verified

> All complete printed programs compiled with javac --release 8 and ran. GUI launchers reported “No display: GUI not opened.” FareCheck and ThemeCheck produced the printed console results. Separate headless component checks exercised listener actions, invalid-input recovery, background completion/interruption, and an EDT action while the worker was waiting. Window appearance, physical mouse clicks, keyboard navigation, resizing, and closing could not be observed: the display server was unreachable and no virtual display was installed.

Course history: Week 11 classes and objects; Week 12 four OOP pillars; Week 13 exceptions; Week 14 collections; Week 15 file handling; Week 16 threads. Catch-up: ask the teacher for the retained source files; do not rewrite the model.

Quick retrieval | Cover the lessons first

1. What class is the window? What class reads one line of input?
2. Which method registers a button listener?
3. Why can a field containing “5” not be used directly as a number?
4. Why is InvalidDistanceException caught separately?
5. Which thread runs sleep in Step 5? Which changes the sign?

Check: JFrame / JTextField; addActionListener; getText returns String; its existing parent is RuntimeException, not IllegalArgumentException; the worker sleeps and the EDT changes the sign.

Opened university references | Short labels used throughout

[1 NTU] Nanyang Technological University — GUI Programming. Brief excerpt: “event-dispatching thread”. Open source

[2 HWS] Hobart and William Smith Colleges — The Basic GUI Application. Brief excerpt: “independent window”. Open source

[3 MIT] Massachusetts Institute of Technology — Recitation 7: Swing, Frame Customization, Events. Brief excerpt: “Event listeners are registered”. Open source

[4 CMU] Carnegie Mellon University (hosted Java Tutorial) — General Rules for Using Swing Components. Brief excerpt: “Support for layout”. Open source

[5 NUS] National University of Singapore (hosted Java Tutorial) — Threads and Swing. Brief excerpt: “The Single-Thread Rule”. Open source

References checked during preparation. CMU and NUS host older Java Tutorial material; use their cited component/thread concepts with this module’s Java 8 startup pattern. Detailed layouts, advanced controls, menus, and market analysis are outside this approved Week 17 split. The linked Weeks 17–21 course map could not be opened.

FRIDAY | Own-theme performance task and presentation

Definition: extend your Week 12 theme with an interface that reuses its objects and rules.

> **ANALOGY:** Build a clear service counter for your theme's user.

Start Thursday after Step 5; present Friday. Use your own theme. Simulate a slow job on a worker if needed; explain the real job it represents.

Requirements checklist

☐ Extend your own theme; retain its model classes.

☐ JFrame: clear title and working close operation.

☐ JLabel, JTextField, JButton, registered ActionListener.

☐ Visible labels, readable feedback, keyboard access.

☐ EDT reads input and calls an existing model method.

☐ Validate numbers, ranges, blanks, unknown entries.

☐ Catch expected errors; allow correction without restarting.

☐ Create and update interface controls on the EDT.

☐ Slow job on a worker; prevent duplicates; restore controls.

☐ Four tests with expected/actual results; present own code.

Plan before you type | Teacher checks first

My plan	Write your choice
Theme / intended user	_____
Existing model + method	_____
Inputs + rules	_____ _____
Result / polite errors	_____
Slow job + safe update	_____ _____

STEP 6 Reuse a theme object

Optional: add ThemeCheck.java; run java ThemeCheck. A Swing listener can read a quantity field, call the model's total method, and display its result or caught error.

ThemeCheck.java

```
public class ThemeCheck {
  static class StoreItem {
    private int price;
    StoreItem(int price) { this.price = price; }
    int total(String text) {
      int qty = Integer.parseInt(text.trim());
      if (qty < 1 || qty > 1000) {
        throw new IllegalArgumentException(
          "Use quantity 1 to 1000.");
      }
      return price * qty;
    }
  }
  public static void main(String[] args) {
    StoreItem rice = new StoreItem(55);
    for (String s : new String[]{"2","0","abc"}) {
      try {
        System.out.println("P" + rice.total(s));
      }
      catch (IllegalArgumentException e) {
        System.out.println(e.getMessage());
      }
    }
  }
}
```

Observed console result:

```
P110
Use quantity 1 to 1000.
For input string: "abc"
```

Two 3-hour days

Thursday: 90 min Step 5 teaching/checks and strip; 30 min lab; 60 min own-theme plan and first action. Get the plan checked before coding.

Friday: 90 min retrieval, repairs, peer tests, teacher check, rehearsal; 90 min presentations/slip. Large classes use simultaneous circles of up to 15; peers record evidence and the teacher rotates.

Testing evidence | At least four ways

Test / expected	Actual result + pass or fix
Valid input / correct model result	_____ _____
Bad input / clear message	_____ _____
Corrected input / usable again	_____ _____
Other action during slow job / responds	_____ _____

Also test blanks and a boundary. Record exact inputs and outputs; allow correction without a crash. Report unverified action checks if no display is available.

Self-check before presenting

Model + listener	Errors + recovery	EDT + worker	Four tests
Yes / Not yet	Yes / Not yet	Yes / Not yet	Yes / Not yet

Five-minute presentation

1. Introduce the theme and user (30 seconds).
2. Run the normal action (1 minute).
3. Prove invalid input, recovery, and responsiveness live (2 minutes).
4. Point to the model call, listener, worker, and EDT update (1 minute).
5. Answer a question (30 seconds).

Prepare for these questions

- What existing model method did you reuse?
- Which line connects the button to its listener?
- How do you refuse invalid input and recover?
- Which code runs on the EDT, and which runs elsewhere?
- What observation proves your interface stays responsive?

Scoring rubric | 50 points

Criterion	Pts	Score
Existing theme and model reuse	8	___
Working Swing controls and listeners	10	___
Validation, correct results, and recovery	10	___
EDT startup, background work, safe updates	10	___
Clear labels, keyboard access, testing evidence	6	___
Presentation and answers using own code	6	___
TOTAL	50	___

```
// Template fragment: fill blanks, add catches.
result.setText("Result: " + ___.___(___));
```

Sources [1 NTU] [2 HWS] [3 MIT] [4 CMU] [5 NUS] (p5). Key validated facts: An interactive interface needs registered listeners [MIT] [HWS], accessible controls [CMU], and safe interface-thread access [NUS].

> **LIPLET · FRIDAY** After Step 6 on page 6, answer the FRIDAY strip (top of page 8) in pen. Tear off only today's strip and submit it.

LIPLET SHEET A · Print single-sided · Detach only today's strip

WEDNESDAY · NOV 11 Reuse the Vehicle Program: Valid Input and Fare Results

Answer after Step 4 on page 3.
Name: _____ Section: _____ Score: _____ / 15

3.1 Define (5 pts): In Steps 3–4, what is validation? Name two distance inputs the program refuses.

3.2 Explain (5 pts): In Step 3, why call Vehicle.quoteFare instead of writing the fare formulas in actionPerformed?

3.3 Connect (5 pts): In Step 4, identify the checked exception for ZZZ and the unchecked exception for 0 km. How does Week 13 help the interface recover?

CUT ON TUESDAY · DETACH THE TUESDAY STRIP BELOW THIS LINE

TUESDAY · NOV 10 A Click Has a Job: Buttons, Text Fields, and Listeners

Answer after Step 2 on page 2.
Name: _____ Section: _____ Score: _____ / 20

2.1 Define (5 pts): In Step 2, what is an event and what does an ActionListener do?

2.2 Explain (5 pts): Trace Step 2 from clicking Show to changing result. What happens if addActionListener(this) is removed?

2.3 Identify (5 pts): Write the class or method: (a) editable text; (b) clickable control; (c) reads field text; (d) changes label text.

2.4 Connect (5 pts): Step 2 implements ActionListener and overrides actionPerformed. Connect this to Week 12 interfaces and polymorphism.

CUT ON MONDAY · DETACH THE MONDAY STRIP BELOW THIS LINE

MONDAY · NOV 9 From Console to Window: Swing and Safe Startup

Answer after Step 1 on page 1.
Name: _____ Section: _____ Score: _____ / 15

1.1 Define (5 pts): What is Swing? In Step 1, what do JFrame and JLabel each do?

1.2 Explain (5 pts): In Step 1, why is interface creation inside SwingUtilities.invokeLater? Name the thread that does it.

1.3 Connect (5 pts): Step 1 keeps Vehicle.java and its subclasses. How does reusing objects connect to Weeks 11 and 12?

FRIDAY · NOV 13 Your Own Theme: Swing Performance Task and Presentation

Answer after Step 6 on page 6 and your presentation.
Name: _____ Section: _____ Score: _____ / 20

5.1 Point (5 pts): In your own theme program, name one model object. Write or identify the line that registers a listener and the line that safely updates the interface.

5.2 Evaluate (5 pts): Defend one input rule and your choice of background work. What would fail if the slow job ran inside the listener?

5.3 Apply (5 pts): A school canteen must calculate an order while loading a saved menu slowly. Which work belongs on the event dispatch thread, and which belongs on a background thread? Explain.

5.4 Reflect (5 pts): Compare your program before and after Swing. Describe one failed test, how you fixed it, and one improvement you would make next.

CUT ON THURSDAY · DETACH THE THURSDAY STRIP BELOW THIS LINE

THURSDAY · NOV 12 Keep the Window Responsive: Background Work and Safe Updates

Answer after Step 5 on page 4.
Name: _____ Section: _____ Score: _____ / 15

4.1 Define (5 pts): In Step 5, how is the background thread different from the event dispatch thread?

4.2 Explain (5 pts): In Step 5, why disable Check ready and use invokeLater to restore it? Why must the listener return quickly?

4.3 Connect (5 pts): Relate ReadyTask to Week 16 Runnable. Why would joining its thread inside actionPerformed freeze other interface actions?

Week 17 CodeShare practice additions

Use this appendix with the original module. Original lesson pages and sample listings remain unchanged. Text backups and data remain unchanged in the supporting-files folder.

Console checks exercise component state, layout or event behavior without opening a window. The message No display: GUI not opened. alone never completes an alteration. Check the actual interface on a school computer; browser GUI rendering and interaction remain unverified.

Run the unchanged sample first. Sample verified records the demonstration only. Make Your task changes, then compare the complete Expected task output. A successful alteration earns Completed. Keep module work, personal work and submitted versions separate. Paper liplets remain teacher assessed.

Step 1 alteration

Sample: java-2-w17-d1-s1 Task: java-2-w17-d1-s1-alter-v1

Add a third JLabel showing "Capacity: 20" to TerminalPanel. Print the actual number of panel controls.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
System.out.println("Panel controls: " + new TerminalPanel().getComponentCount());
```

Expected task output

```
Panel controls: 3
No display: GUI not opened.
```

Step 2 alteration

Sample: java-2-w17-d2-s2 Task: java-2-w17-d2-s2-alter-v1

Change the initial plate field to " TRC 456 ". Invoke the existing Show handler and read its actual result label; spaces must be trimmed.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
TerminalPanel panel = new TerminalPanel();
panel.actionPerformed(null);
System.out.println(((JLabel) panel.getComponent(3)).getText());
```

Expected task output

```
Plate: TRC 456
No display: GUI not opened.
```

Step 3 alteration

Sample: java-2-w17-d3-s3 Task: java-2-w17-d3-s3-alter-v1

Change the distance field from 5 to 8. Invoke the Quote fare handler and print its actual result label.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

```
TerminalPanel panel = new TerminalPanel();
panel.actionPerformed(null);
System.out.println(((JLabel) panel.getComponent(5)).getText());
```

Expected task output

```
NXA 123: P21.00
No display: GUI not opened.
```

Step 4 alteration

Sample: java-2-w17-d3-s4 Task: java-2-w17-d3-s4-alter-v1

In FareCheck change only the BUS 789 distance from 4 to 8. Keep the invalid-input cases and their exception checks.

Expected task output

```
NXA 123: P15.00
TRC 456: P40.00
BUS 789: P35.00
NYB 321: P17.00
NXA 123: P15.00
VehicleNotFoundException: Unknown plate: ZZZ
NumberFormatException: For input string: "abc"
InvalidDistanceException: Use a finite km above 0.
InvalidDistanceException: Use a finite km above 0.
InvalidDistanceException: Use a finite km above 0.
```

Step 5 alteration

Sample: java-2-w17-d4-s5 Task: java-2-w17-d4-s5-alter-v1

Exercise the interrupted-worker path. In Main declare main throws Exception. Start the Swing event queue with an empty invokeAndWait call. Create a disabled button and status label, start and then interrupt a ReadyTask worker, join it, then flush the Swing event queue before reading the actual status and button state.

Before the existing headless check in `Main.java`, add this console check. Keep the original sample message and the window code.

SwingUtilities.invokeAndWait(() -> {});
JLabel status = new JLabel("Not checked");
JButton button = new JButton("Check ready");
button.setEnabled(false);
Thread worker = new Thread(new ReadyTask(status, button));
worker.start();
worker.interrupt();
worker.join();
SwingUtilities.invokeAndWait(() -> {
System.out.println(status.getText());
System.out.println("Button enabled: " + button.isEnabled());
});

Expected task output

```
Check interrupted; try again.
Button enabled: true
No display: GUI not opened.
```

Step 6 alteration

Sample: java-2-w17-d5-s6 Task: java-2-w17-d5-s6-alter-v1

Change the StoreItem price from 55 to 60 and its valid test quantity from 2 to 3. Keep zero and abc as rejected inputs.

Expected task output

```
P180
Use quantity 1 to 1000.
For input string: "abc"
```